An Explainable Ensemble Learning Framework for Early Patient Risk Prediction Using Two Healthcare Datasets

Arya Jain
Department of Computer Science and Engineering
Symbiosis Institute of Technology
Nagpur, India
arya.jain.btech2023@sitnagpur.siu.edu.in

Vismayaa Rajan
Department of Computer Science and Engineering
Symbiosis Institute of Technology
Nagpur, India
vismayaa.rajan.btech2023@sitnagpur.siu.edu.in

Abstract* — *In this work, we took a close look at just how well machine learning algorithms can do on real-world problems in the healthcare space using two kinds of datasets: the Diabetes Health Indicators from the BRFSS survey, and a second one - the Diabetes 130-US Hospitals dataset. Our main goals were to see how well we could predict whether someone has diabetes (in different levels of severity) and to make a rough guess at how likely a patient is to get sent back to the hospital within a month after a visit. To do this we ran four different models - Logistic Regression, Random Forest, Extra Trees, and a weighted voting system - and split the data up 80/20 to train the models.

Keywords- Healthcare Analytics, Diabetes, Forecasting, Hospital Readmissions, Ensemble Learning Class Imbalance, Putting AI into words, Random Forest

I. INTRODUCTION

First off, we're talking about figuring out whether someone is healthy, on the verge of getting diabetes, or is actually dealing with diabetes. And then we've got the second part, which is trying to guess how likely a hospital visit is to result in a readmission within 30 days. We split these two datasets up because they involve completely different groups of people, require different types of information, and are looking at slightly different outcomes. What makes our project stand out is that we're going to be running the same set of models on both datasets and then picking a cut-off point to use when trying to predict those tricky readmissions.

Predictive analytics can be super useful in population health and hospital discharge predictions - but in practice, healthcare data is basically a mess. There's tonnes of different variables hanging around and its not always clear cut cause and effect.

This means that, inevitably, a classifier is going to get really good at predicting the most common outcome - but will probably miss those patients who really need our attention. Our evaluation metrics cant just be based on accuracy anymore - we need to get a better sense of how our models are performing by taking a look at stuff like precision, recall, f1 score, roc auc & confusion matrices. We've got a bit of a challenge on our hands because were tackling two separate prediction problems using two very different datasets here. The first one is all about figuring out whether someone is healthy, has pre diabetes or diabetes - meanwhile were trying to work out which hospital visits are likely to end in a readmission within 30 days. We're treating these two datasets as two completely separate problems because they're looking at different populations, have different features, and are targeting entirely different goals. What makes this project stand out is that were going to be consistently comparing models, and then selecting a threshold for that notoriously tricky readmission problem Healthcare Data Analysis revolves around taking the mountains of standard health data and distilling it down to actual useful information that can actually help prevent issues, identify patients at risk, and come up with better service plans. But when the outcome classes are seriously skewed , the variables are a mess of numbers & categories rather than some nice tidy set, and the cost of missing one case is so high compared to the cost of putting in a bit more effort to double check - then simply relying on one measure of model accuracy just won't cut it. We need to use a bunch of different metrics - precision, recall, F1-score, ROC-AUC and a good old fashioned confusion matrix - to get a better, more honest picture of how well our models are actually working, and to make some informed decisions about what next steps we should take.

The paper we're talking about looks at 2 separate tasks. One of them is all about using population health data to try & figure out someones diabetes status. The other is looking at using hospital data to try and identify who's at risk of early readmission. We decided to treat these as two separate tasks because it just wouldn't make sense to try and mash them together into one big modelling dataset. Actually , an experimental protocol is put into place for each data set one at a time , allowing us to compare the candidate classifiers fairly and keep the meaning behind each result consistent.

The questions we're trying to answer are: (1) how do Logistic Regression, Random Forest, Extra Trees and an ensemble of classifiers with weighted voting do on each problem; (2) is this ensemble better at certain metrics than any one of the individual learners; (3) how does selecting a certain threshold affect the

prediction of early readmissions; and (4) what features are most important in our learned 'hospital tree' model? This study is an exploration of the way different models behave, but we're not suggesting a new way to make clinical decisions.

II. DATASETS AND TARGETS

Dataset 1 - Diabetes Health Indicators (BRFSS derived): The CSV file had 253,680 rows of data and 22 columns.

The target - Diabetes_012 - had 3 classes: people with diabetes, people with pre-diabetes and people with no sign of diabetes. Our analysis filtered out 23,899 duplicates and ended up with 229,781 rows and 21 predictors to work with from the original 22.

Dataset 2 - Diabetes 130-US Hospitals: We also had another CSV file to contend with - which just so happened to be a whopper, 101,766 hospital encounters and 48 columns to sift through. The readmitted field originally was a right old mess with labels like NO, >30, and <30. We tidied all that up by turning it into a simple binary target variable: If a patient got readmitted within 30 days they got a 1, and everyone else got a 0. We also chucked a couple of columns from the list of predictors that just weren't doing us any favors - encounter_id and patient_nbr - out went the useless ones. And as for the whopping 374,017 missing values - we swapped the '? ' symbols out for good old NaNs.

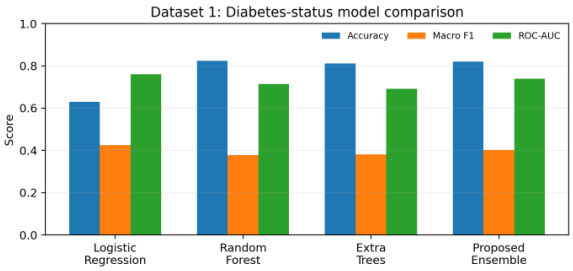


Fig. 1. Model comparison for Dataset 1 - the scores are based on the notebook's saved test-set comparison.

TABLE I. DATASET DESCRIPTION, TARGET DEFINITION AND SPLIT SIZES.

Dataset	Original records	Columns	Cleaning / target	Model features	Train / test
Diabetes Health Indicators	253,680	22	Drop 23,899 duplicates; 3-class Diabetes_012	21	183,824 / 45,957
Diabetes 130-US Hospitals	101,766	48	Map '?' to missing; <30 = 1, other labels = 0	47 (IDs excluded)	81,412 / 20,354

Dataset 1: Diabetes Health Indicators: The CSV data for this one had a huge number of records - 253,680, and 22 columns to boot. Once we'd tidied up the data a bit, it boiled down to three clear categories for outcomes: 0 - no diabetes, 1 - prediabetes, and 2 - diabetes. But before we could do that, we had to chop off 23,899 duplicate rows - once we'd done that we were left with 229,781 records. That trimmed down the extra rows but didn't exactly address the issues that come with working with survey based data. And as anyone who's worked with this kind of data will tell you, one thing it's not short on is limitations. The classes in this dataset are also pretty heavily skewed - 213,703 records in class 0, 4,631 in class 1 and a whopping 35,346 in class 2. With prediabetes making up such a small minority, we really need to be careful and use measures that take that into account.

Dataset 2: Diabetes 130-US Hospitals: This hospital file is a big one - 101,766 encounters to navigate and a massive 48 columns to get our heads around. The readmitted variable has one of the most confusing sets of labels I've ever come across - it's NO, >30, and 30. But for the sake of this project, we're going to take a much simpler view: if the label is 30 its a yes, if it's NO or >30 then its a no. So it's not about whether the patient got readmitted a whole month later, but whether they got readmitted within the 30 day window. Of the 54,864 records with the original labels, 35,545 were originally in the >30 category and 11,357 were in the 30 category - and that works out as roughly 11.2% of the 'no' category, and to be honest, that's not very big compared to the rest. All the missing values in the form of ? in the hospital dataset were treated as missing - and it looks like there's a whopping 374,017 of them in the supplied file. To stop the model picking up record-specific stuff, we deliberately excluded fields like encounter_id and patient_nbr from the predictors. And because of that, we may end up splitting some patients records between different groups, a problem that should get looked at in more detail down the line in a patient grouped validation.

III. PREPROCESSING AND MODELS

Both datasets were split up using Train/Test Split with a test size of 0.20 and a random seed of 42 to ensure things are kept consistent. The feature imputation in the pre-processing pipeline is handled by scikit-learn. The numerical features were treated with the median of the training data, then standardised. Categorisation features on the other hand used the most frequent category from the training set and were one-hot encoded - any unknowns were simply ignored. This pipeline helps prevent leakage by allowing the model to learn its parameters as it goes.

Four different classifiers were compared for each task: number one was a Logistic Regression with a high max iteration count of 1000 and class weights balanced to steer it away from bias; number two was a Random Forest with 100 trees, balanced class weighting and parallel execution; number 3 was an Extra Trees model with a 100 tree depth and balanced class weighting; and number 4 was a soft voting ensemble with weights of 1,2,2 for the respective Logistic Regression, Random Forest and Extra Trees models. For the hospital task, a sixth model - another Random Forest - was added with a balanced subsample class

weight and min leaf sample size set to 2. One the hospital task we also used a Random Forest with 'balanced subsample' weighting - but then we also tried a different min leaf sample size.

To put soft-voting into practice we use a weighted average of the component probabilities - the class with the highest ensemble probability was the one we chose. That weighted average looks like this: P_(ensemble) (y = k | x) = (P_lr(k|x) + 2 * P_rf(k|x) + 2 * P_et(k|x)) / 5.

The data is split, 80% for training 20% for testing, with a random number generator seed of 42. This is done so that the two parts of the data have the same class distributions and we end up with a more realistic benchmark for the minority class. Both the data pre-processing steps are handled by scikit-learn so that only the training data is used to fit the imputer, scaler and encoder. If we do the pre-processing before splitting the data, we run the risk of introducing the test results into our model and getting overly optimistic scores.

In terms of numeric predictors, we fill in missing values with the median - this is worked out from the training set. StandardScaler then takes a numeric value and turns it into a z score - this is the value subtracted by the mean and then divided by the standard deviation, based on the training set. For categorical predictors, missing values are replaced with the most common category that turned up in the training data. Our OneHotEncoder is set up to ignore any unknown categories that might show up when the model is put into practice and results in the creation of binary indicator columns. If there are any categories that show up in the test data but not in the training data, this prevents any errors popping up at inference time. We stick to the same pipeline design for all of our model comparisons.

Logistic Regression gives us a pretty simple linear baseline to work with - it estimates the probability of a particular class based on a linear combination of the features and so is a good benchmark because it's decision structure is smaller than a tree based model. It's been implemented with a maximum number of iterations of 1000 and class weights set to be equal. Random Forest then builds a bunch of decision trees and uses their predictions to come up with a final result - it uses 100 trees, is seeded with 42, has class weighting, and uses all available cores. Extra Trees does a similar thing to Random Forest but introduces a bit more randomness into the process of selecting splits. Both tree based methods are able to pick up complex relationships and interactions between features that might get missed by a linear classifier.

IV. EVALUATION METRICS

Accuracy is the overall number of correct predictions out of the total number of predictions made. Precision is how often you actually get a true positive result when you predict one. Your recall, or sensitivity, is how often you actually do catch all the real positives. F1 is the middle ground between precision and recall, giving you some idea of how well your model is doing on both fronts. Specificity is a good indicator of whether your model is good at keeping track of what it knows isn't true, keeping a handle on the negatives just as tightly.

F1 is the harmonic mean of precision and recall which means its a number that combines the two to give a better idea of a model's performance. Macro-averaged metrics for precision, recall and F1 give each class (in the case of the multiclass diabetes comparison) the same weight and are used kind of like a broad brush to get an idea of how well a model is doing. You can find all of these metrics in the notebook's standard model table for both datasets. The separate table for the tuned readmission model gives just the positive class metrics because for the readmission task, it turns out that most of the time when someone’s not supposed to be readmitted (less than 30 days) they’re not. That’s only about 11.2% of the time. So a high accuracy score might look pretty good but they could be getting by on missing lots of early readmissions.

Accuracy is calculated as (the number of true positives plus true negatives) divided by the total (true positives plus true negatives plus false positives plus false negatives). Precision is the number of true positives divided by (true positives plus false positives). And your recall, or sensitivity, is the number of true positives divided by (true positives plus false negatives). F1 is a special number that combines the two - its calculated as 2PR divided by (P+R) which is a measure of both precision and recall. Specificity is the number of true negatives divided by (true negatives plus false positives) which gives you a sense of how well your model is keeping an eye on things that arent supposed to be true. ROC-AUC ,on the other hand, is a measure of the models ability to tell apart the true positives from the true negatives - its a kind of ranking system that shows how well the model can spot the real positives when they come up. With so many different metrics out there no single one is ever going to be enough to tell the whole story - especially when you have false negatives and false positives that have different consequences in the real world.

ROC-AUC and the confusion matrix are used as threshold based measures. As seen in the original readmission ensemble, the model could have moderate ranking discrimination but a poor default threshold. On the other hand, setting a lower threshold for what counts as a positive encounter might turn up more instances of people having a good time , but it's likely to pull in a lot of false alarms too. In a real-world service, figuring out just where to set that threshold would depend on a bunch of factors - how well interventions actually work, how much time and effort goes into trying to reach people, and what the cost is of ignoring the early warning signs of readmissions.

V. RESULTS

A. Dataset 1—Diabetes Status

Table II lays out how the 4 classifiers did on the held-out test we set aside. And it was the Random Forest algorithm that came out on top with a very respectable accuracy of 82.31%. But our proposed ensemble model wasn't far behind, racking up an accuracy of 82.197%. What caught my eye though is that the Logistic Regression model actually did a lot better when it came to Macro F1 (0.4257) and ROC-AUC (0.7607). In contrast, our proposed model did okay on the Macro F1 front but a pretty disappointing 0.4030 - and it's ROC-AUC of 0.7398 didn't even quite make up for that. So, while the proposed ensemble model had some strengths, it wasn't the top dog when it came to the more class-balanced measures.

TABLE II. DATASET 1 TEST-SET MODEL COMPARISON (MACRO-AVERAGED PRECISION, RECALL AND F1).

Model	Accuracy	Macro precision	Macro recall	Macro F1	ROC-AUC
Logistic Regression	0.6292	0.4475	0.5135	0.4257	0.7607
Random Forest	0.8231	0.4376	0.3734	0.3771	0.7142
Extra Trees	0.8112	0.4189	0.3757	0.3805	0.6920
Proposed Ensemble	0.8197	0.4405	0.3945	0.4030	0.7398

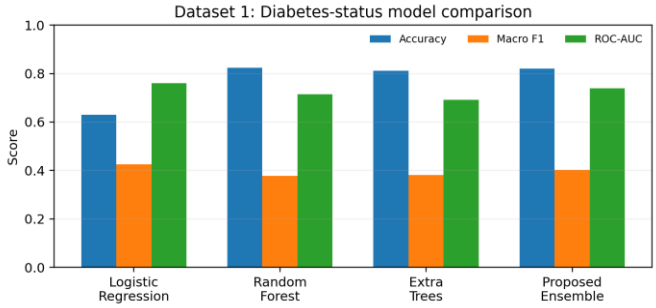


Fig. 2. Diabetes-status accuracy, macro F1 and ROC-AUC by classifier.

Similarly, the class level F1 for the proposed ensemble is 0.90 for no diabetes, 0.00 for prediabetes, and 0.31 for diabetes. There were 38,012 cases of no diabetes, 926 cases of prediabetes, and 7,019 cases of diabetes in the test set. The high overall accuracy is for the majority class and it is quite difficult to detect prediabetes in the models. The three-fold cross-validation macro F1 is higher for Logistic Regression (mean 0.4211, standard deviation 0.0019) over the weighted ensemble (mean 0.4017, standard deviation 0.0004).

B. Dataset 2—30-Day Readmission, Default Threshold

Table II summarizes the held out tests for the four classifiers. Random Forest yielded the highest accuracy (0,8231), followed by the proposed ensemble (0,82197). But, Logistic Regression had the highest macro F1 (0,4257) and ROC-AUC (0,7607). The ensemble's macro F1 was 0.4030, whereas its ROC-AUC was 0.7398, so it was not superior to Logistic Regression in the more class-balanced measures.

TABLE III. DATASET 2 DEFAULT-THRESHOLD MODEL COMPARISON. MACRO METRICS SUMMARIZE BOTH BINARY CLASSES; SPECIFICITY WAS CALCULATED BY THE NOTEBOOK FOR THE ORIGINAL RESULTS.

Model	Accuracy	Macro precision	Macro recall	Macro F1	ROC-AUC
Logistic Regression	0.6418	0.5430	0.6011	0.5095	0.6440
Random Forest	0.8887	0.7602	0.5024	0.4757	0.6511
Extra Trees	0.8888	0.7255	0.5064	0.4843	0.6345
Proposed Ensemble	0.8892	0.7545	0.5079	0.4873	0.6653

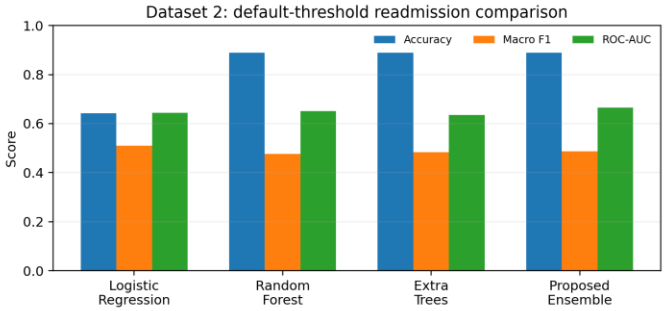


Fig. 3. Default-threshold readmission model comparison. High accuracy is not equivalent to strong positive-class detection.

TABLE IV. POSITIVE-CLASS METRICS FROM THE NOTEBOOK'S COMPARISON OF THE ORIGINAL ENSEMBLE AND CLASS-WEIGHTED RANDOM FOREST.

Default-threshold model	Accuracy	Early-readmission precision	Early-readmission recall	Early-readmission F1	ROC-AUC
Original Ensemble	0.8892	0.6190	0.0172	0.0334	0.6653
Balanced Random Forest	0.8500	0.2689	0.2004	0.2296	0.6710

C. Dataset 2—Threshold-Tuned Balanced Random Forest

The search for the validation threshold was 0.40; its precision was 0.1795, recall was 0.5096 and F1 was 0.2655. On the untuned set of test data, the model with the threshold was 0.6876 with precision 0.1856, recall 0.5310, F1 0.2750 and ROC-AUC 0.6671. As with the original group, recall was increased substantially and precision and accuracy decreased. It is in fact a sensitivity-false-alarm trade-off rather than an overall increase.

TABLE V. FINAL HELD-OUT TEST RESULTS FOR THE THRESHOLD-TUNED BALANCED RANDOM FOREST. ALL NUMBERS ARE FROM THE NOTEBOOK'S LAST CELL OF EVALUATION AND FROM THE FILE FINAL_READMISSION_TEST_METRICS.CSV.

Threshold	Accuracy	Precision	Recall	F1	ROC-AUC
0.40	0.6876	0.1856	0.5310	0.2750	0.6671

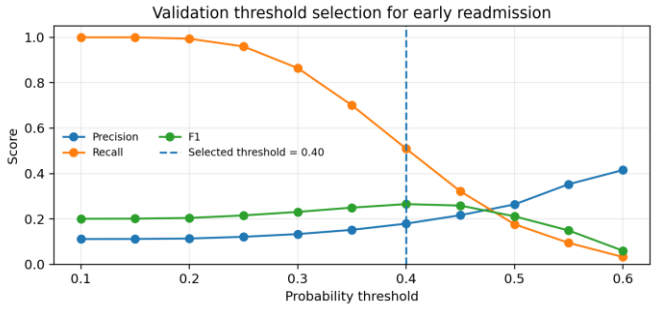


Fig. 4. Validation precision, recall and F1 across candidate thresholds; the selected operating threshold is 0.40.

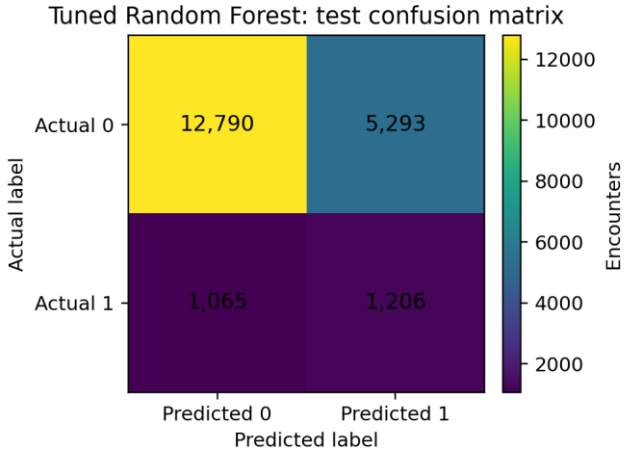


Fig. 5. Test confusion matrix at 0.40: TN=12,790, FP=5,293, FN=1,065, TP=1,206.

In addition, the model missed 1 065 of the 2,271 early readmission encounters in its confusion matrix. Of that total, 5,293 were false positives. This volume could present a problem in a real-world discharge planning situation. The best threshold for the model should be based on available intervention resources, the cost of missed readmissions and the cost of unnecessary follow-up.

D. Hospital Feature Importance

The notebook's Random Forest feature-importance output ranked number of laboratory procedures (0.0472), number of medications (0.0446), number of inpatient visits (0.0383), and time in hospital (0.0358) as the four highest-ranking features. Discharge status, number of diagnoses, number of procedures, admission type/source, and outpatient visits also had importance. These scores represent how the fitted trees split, but are not causal estimates nor does this indicate that changing a feature would change readmission risk.

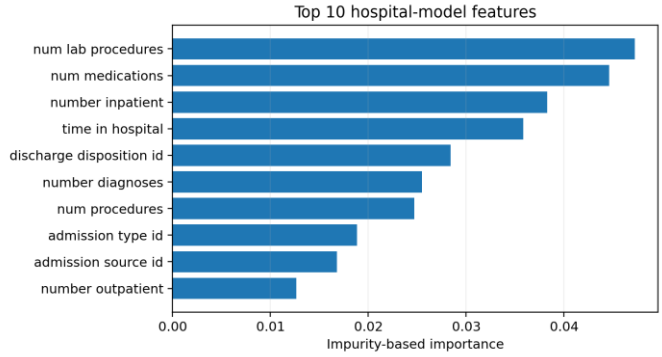


Fig. 6. Top ten hospital feature importances from the supplied notebook output.

The results are presented separately for the two datasets, as the targets and metric interpretations differ. The notebook's model comparison outputs are presented in Tables II and III. The readmission threshold is reported separately in Table V because it used a different model configuration and decision threshold. Separating these experiments prevents confusion between the ensemble performing at the default threshold and the threshold-tuned Random Forest.

VI. DISCUSSION AND LIMITATIONS

The results clearly show that the performance of the model depends on the goal you're trying to achieve and the specific objective you're aiming for. In the diabetes status case, for example, we found that Logistic Regression surprisingly came out on top when using macro F1 and ROC-AUC scores as our measure, even though Random Forest actually had the highest accuracy - which doesn't always translate to being the best choice. Same thing happened in the readmission case - and that was with our supposedly state-of-the-art ensemble model that

just about missed all the positive cases to start off with , only to start doing better once we tweaked the threshold a bit - but in the process we ended up sacrificing some precision & had a bunch of false alarms to deal with as a result So lets not get too carried away assuming a multi-classifier ensemble is always going to outshine the rest.

The two datasets we're working with have their own sets of issues The diabetes data, for instance, is based on survey responses - which isn't exactly ideal as you'd get a much clearer picture of someone's condition if you had actual test results . And to make matters worse, there's only a tiny little prediabetes category in there, which makes it a real pain to work with And then theres the hospital data - its actually just a collection of individual patient records - but its got all sorts of gaps in it, & even then it might not give you a reliable picture of the patient's current situation. We're doing a split of the data at the encounter level, which means if someone's got multiple encounters they might just end up in both the training & test sets , and that just isn't ideal - it could end up skewing the results. All this means that any real analysis should ideally be done at the patient level, and should involve putting patients into groups, checking that the model is performing well over time, and even some checks to see if the model is performing equally well for different groups of patients.

It's also worth noting some of the differences in the metrics that we're reporting across the notebook - we're doing macro-averaged metrics in this main comparison table, but in this specific test table we're doing positive-class metrics. This matters because you can't just compare macro F1 directly to positive-class F1, so to avoid that confusion we're keeping them in separate tables as we are now.

The diabetes experiment is a good illustration of just how tricky it can be when you're trying to select the best model when you've got a lot of classes that are all very imbalanced. Random Forest ended up having the highest accuracy, but in the end Logistic Regression was actually the better model when you looked at either macro F1 or ROC-AUC scores. And that proposed ensemble of theirs - it didn't even come close when it came to those balanced class measures that actually mattered.

Plus, the prediabetes class in particular turned out to be really tough to detect - which means that any model that's supposed to be able to identify this class is still a long way off from being something that's reliable for real-world use. Any future work on this should really include doing some more advanced analysis like class-specific recall, per-class confusion matrices, doing some calibration checks, and even evaluating on an independent dataset to be sure the model is holding up.For readmission, the original ensemble's high accuracy came hand-in-hand with an extremely disappointing early-readmission recall rate. This kind of behaviour suggests that the default classification threshold just wasn't well-suited to picking out those hard-to-spot positive cases. The threshold-tuned Random Forest did manage to bump up recall to 0.5310, but at the cost of precision plummeting to 0.1856, and amongst other things the test confusion matrix showed a whopping 5,293 cases that turned out to be nothing but false alarms. You could argue that this was a trade-off worth making, but deciding that is going to require a whole lot more than just crunching the F1 score. If you genuinely want to get a feel for whether this approach is going to work then what you really need to do is put it through its paces with an operational evaluation - which is basically just a matter of figuring out how many alerts you're likely to be getting each day, how much spare capacity you've got available to handle care management, and what the knock-on effects would be in cases where you get some things right and some things wrong. The hospital feature-importance values do a decent job of highlighting some obvious things like the number of lab tests, the number of medications a patient is taking, the number of times they've been in for in-patient visits, and how long they've actually been in the hospital. It's really not too hard to see why these make the list, as they are all indicators you'd expect to see in someone who's had a pretty complicated time. On the other hand, built-in tree importance is basically just telling you a bit about the way the inputs are structured, and it can be easily skewed by the way the feature is constructed or the kinds of structures it gets correlated with - let alone trying to use it to prove any kind of causality, give you a patient-specific explanation, or even use it as some kind of actual clinical recommendation. And on top of that, the fact that the diabetes and hospital datasets are different in nearly every way (population, time frame, even actual measurements) really doesn't help the situation at all when it comes to trying to generalize from this one particular study. You've got a bunch of other limitations on top of that - basically you've got historical data, incomplete records, repeat-patient-sensitivity issues due to how the encounter-level split was done, hyperparameters that were fixed from the start, and no prospective or external validation at all. Its no real surprise but the report did at least use a validation set to figure out a threshold to use - although even so there's still a fair bit of work to put in on things like calibration, interval confidence, how the model performs on different subgroups, patient groups and also doing some proper validation over time. And if you actually want to put this in a real-world care pathway then you'll need a lot of guidance from clinicians and some proper evaluation before you even think about sticking your neck out and putting it in real world use.

VII. CONCLUSION

We took 4 different classifiers - Logistic Regression, Random Forest, Extra Trees and a weighted soft-voting ensemble — and pitted them against two different datasets that were struggling with hospital readmission issues . We put them through their paces, measuring up how well they did on macro F1 and ROC-AUC scores. We also threw in a few ensemble tricks for good measure. Honestly, even though the ensemble didn't come out

on top against them all, Logistic Regression impressed us by delivering a pretty strong performance in predicting diabetes status using those macro F1 and ROC-AUC scores - in fact the ensemble didn't even come close to those results on every measure. But on the hospital readmission front - well, that was a different story altogether - the ensemble trailed behind in all metrics. Although it did manage to squeeze out some small improvement courtesy of taking the default ensemble's pathetic recall rate and turning it around to a fairly respectable 53.10% - although that improvement came at a price: an eye-watering 5,293 false alarms. We're going to say it again and again - and we really can't stress it enough - it's absolutely essential to report metrics in a rock-solid transparent way, make it crystal clear what the targets are and what trade-offs you're making. These models are still purely experimental at this stage and are a long, long way off from being ready to use in real-world clinical decision-making.

So, we decided to go back and see if we could replicate the results using Logistic Regression, Random Forest, Extra Trees and another weighted soft-voting ensemble. And guess what? Once again, Logistic Regression walked away with the top spot for diabetes status classification using macro F1 and ROC-AUC scores on both datasets - while Random Forest outperformed it in terms of overall accuracy.

When it comes to predicting early hospital readmission, the original ensemble actually started to show some promise - with a high accuracy score, at least - but the cost was that it barely managed to detect any cases at all, default setting and all that. Tweaking the threshold on the balanced random forest search brought in rather more positive cases - 53.10% to be exact - but the trade-off was a pretty dismal 18.56% precision rate and a pretty mediocre F1 score of 0.2750, to say nothing of a whole load of unnecessary false alarm noise. So, the upshot of all this was that we came to the conclusion that when it comes to healthcare model evaluation, you really do need to keep a very close eye on how it's going to be used and what the real-life consequences might be. So - and I really hope it's sooner rather than later - we need to start beefing up patient-level validation, using some external validation sets, and take a closer look at the calibration.

REFERENCES

[1] "Predicting 30-Day Hospital Readmission in Patients With Diabetes Using Machine Learning on Electronic Health Record Data," Cureus, vol. 17, no. 4, e82437, 2025, doi: 10.7759/cureus.82437.

[2] "An interpretable machine learning approach for predicting 30-day readmission after stroke," International Journal of Medical Informatics, vol. 174, art. 105050, 2023, doi: 10.1016/j.ijmedinf.2023.105050.

[3] "Machine learning for predicting readmission risk among the frail: Explainable AI for healthcare," Patterns, vol. 3, no. 1, art. 100395, 2022, doi: 10.1016/j.patter.2021.100395.

[4] "An interpretable imbalance ensemble classification method for readmission risk assessment incorporating multi-view perturbation and SHAP analysis," Decision Support Systems, vol. 190, art. 114404, 2025, doi: 10.1016/j.dss.2025.114404.

[5] G. Du et al., "Towards graph-based class-imbalance learning for hospital readmission," Expert Systems with Applications, 2021, doi: 10.1016/j.eswa.2021.114791.

[6] "A Machine Learning Approach for Predicting 30-Day Hospital Readmission in Patients with Diabetes," Healthcare, vol. 14, no. 9, art. 1185, 2026, doi: 10.3390/healthcare14091185.

[7] "An Innovative Ensemble Deep Learning Clinical Decision Support System for Diabetes Prediction," IEEE Access, vol. 12, pp. 106193–106210, 2024, doi: 10.1109/ACCESS.2024.3436641.

[8] "A novel machine learning approach for diagnosing diabetes with a self-explainable interface," Healthcare Analytics, vol. 5, art. 100301, 2024, doi: 10.1016/j.health.2024.100301.

[9] M. Kutlu, T. B. Donmez, and C. Freeman, "Machine Learning Interpretability in Diabetes Risk Assessment: A SHAP Analysis," Computers and Electronics in Medicine, vol. 1, no. 1, pp. 34–44, 2024, doi: 10.69882/adba.cem.2024075.

[10] "Explainable AI Applied in Healthcare: A Case Study of Diabetes Prediction," in Proc. IEEE QRS-C, 2024, pp. 336–344, doi: 10.1109/QRS-C63300.2024.00050.

[11] "Integrating Machine Learning with Explainable AI in Healthcare Analytics for Diabetes Prediction," in Proc. ICEDEG, 2025, doi: 10.1109/ICEDEG65568.2025.11081581.

[12] P. Shah et al., "Predicting cardiovascular risk with hybrid ensemble learning and explainable AI," Scientific Reports, vol. 15, art. 17927, 2025, doi: 10.1038/s41598-025-01650-7.

[13] "Explainable ensemble learning model for cardiovascular disease prediction with feature optimization and data balancing," Discover Computing, vol. 29, art. 44, 2026, doi: 10.1007/s10791-026-09930-0.

[14] "A Robust and Explainable Framework for Predicting Hospital Readmission in Patients with Diabetes," 2026. Available: https://www.mdpi.com/2673-4540/7/10/197.

[15] "Predictive Analytics for Diabetic Patient Care: Leveraging AI to Forecast Readmission and Hospital Stays," 2025. Available through ScienceDirect.